Com S // Cpr E // Math 5250

Numerical Analysis of High-Performance Computing

Instructor: Songting Luo

Lecture 22: Iterative Methods for Solving Linear Systems

Outline

1. Model Equation: Poisson Equation

2. Simple Iteration Methods

Model Equation: Poisson Equation

Heat Equation / Diffusion Equation

Partial differential equation (PDE) for $u(t, x)$

in one space dimension (x) and time (t).

u represents temperature in a 1-dimensional metal rod.

Or concentration of a chemical diffusing in a tube of water.

Heat Equation / Diffusion Equation

Partial differential equation (PDE) for $u(t, x)$

in one space dimension (x) and time (t).

u represents temperature in a 1-dimensional metal rod.

Or concentration of a chemical diffusing in a tube of water.

The PDE is

$$u(t, x)_{,t} = Du(t, x)_{,x,x} + g(t, x)$$

where subscripts represent partial derivatives:

$$u_{,t} := \frac{\partial u}{\partial t} \qquad \text{and} \qquad u_{,x,x} := \frac{\partial^2 u}{\partial x^2}.$$

$D =$ diffusion coefficient (assumed constant in space & time)

$g(t, x) =$ source term (heat or chemical being added/removed)

Heat Equation / Diffusion Equation

Partial differential equation (PDE) for $u(t, x)$

in one space dimension (x) and time (t).

u represents temperature in a 1-dimensional metal rod.

Or concentration of a chemical diffusing in a tube of water.

The PDE is

$$u(t, x)_{,t} = Du(t, x)_{,x,x} + g(t, x)$$

where subscripts represent partial derivatives:

$$u_{,t} := \frac{\partial u}{\partial t} \qquad \text{and} \qquad u_{,x,x} := \frac{\partial^2 u}{\partial x^2}.$$

$D =$ diffusion coefficient (assumed constant in space & time)

$g(t, x) =$ source term (heat or chemical being added/removed)

Also need initial conditions $u(0, x)$ and boundary conditions $u(t, a), u(t, b)$

Steady state heat/diffusion

If $g(t,x) = g(x)$ does not depend on time and if the boundary conditions do not depend on time, then $u(t,x)$ will converge to a steady-state distribution satisfying:

$$0 = Du(x)_{,x,x} + g(x)$$

(by setting $u_{,t} = 0$).

Steady state heat/diffusion

If $g(t,x) = g(x)$ does not depend on time and if the boundary conditions do not depend on time, then $u(t,x)$ will converge to a steady-state distribution satisfying:

$$0 = Du(x)_{,x,x} + g(x)$$

(by setting $u_{,t} = 0$).

This is now an ordinary differential equation (ODE) for $u(x)$.

We can solve this on an interval, say $0 \leq x \leq 1$, with boundary conditions:

$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

Steady state heat/diffusion

If $g(t,x) = g(x)$ does not depend on time and if the boundary conditions do not depend on time, then $u(t,x)$ will converge to a steady-state distribution satisfying:

$$0 = Du(x)_{,x,x} + g(x)$$

(by setting $u_{,t} = 0$).

This is now an ordinary differential equation (ODE) for $u(x)$.

We can solve this on an interval, say $0 \leq x \leq 1$, with boundary conditions:

$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

If we let $f(x) = g(x)/D$, then we arrive at the ODE:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \qquad \text{and} \qquad u(1) = \beta.$$

Steady state heat/diffusion

2-Point Boundary Value Problem:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

Steady state heat/diffusion

2-Point Boundary Value Problem:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

Example: $\alpha = 20$, $\beta = 60$, $f(x) \equiv 0$ (no heat source)

Steady state heat/diffusion

2-Point Boundary Value Problem:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

Example: $\alpha = 20,\ \ \beta = 60,\ \ f(x) \equiv 0$ (no heat source)

Solution:

$$u'' = 0 \quad \Longrightarrow \quad u(x) = a + bx$$
$$u(0) = a = 20$$
$$u(1) = 20 + b = 60 \quad \Longrightarrow \quad b = 40$$
$$u(x) = 40x + 20$$

Steady state heat/diffusion

2-Point Boundary Value Problem:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

Example: $\alpha = 20, \ \beta = 60, \ f(x) \equiv 0$ (no heat source)

Solution:

$$u'' = 0 \quad \Longrightarrow \quad u(x) = a + bx$$
$$u(0) = a = 20$$
$$u(1) = 20 + b = 60 \quad \Longrightarrow \quad b = 40$$
$$u(x) = 40x + 20$$

No heat source $\Longrightarrow$ linear variation in the steady-state

Steady state heat/diffusion

2-Point Boundary Value Problem:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

More interesting example:

Example: $\alpha = 20, \ \beta = 60, \ f(x) \equiv 100e^x$

Steady state heat/diffusion

2-Point Boundary Value Problem:

$$-u''(x) = f(x)$$
$$u(0) = \alpha \quad \text{and} \quad u(1) = \beta.$$

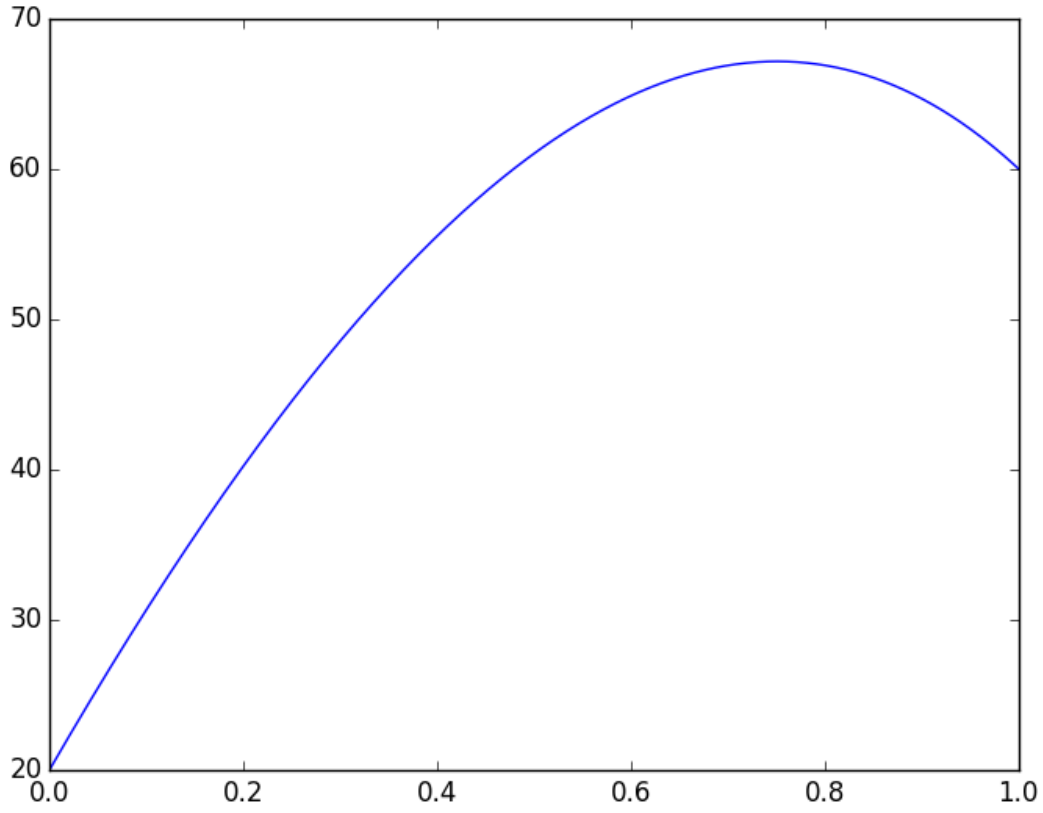
More interesting example:

Example: $\alpha = 20, \ \beta = 60, \ f(x) \equiv 100e^x$

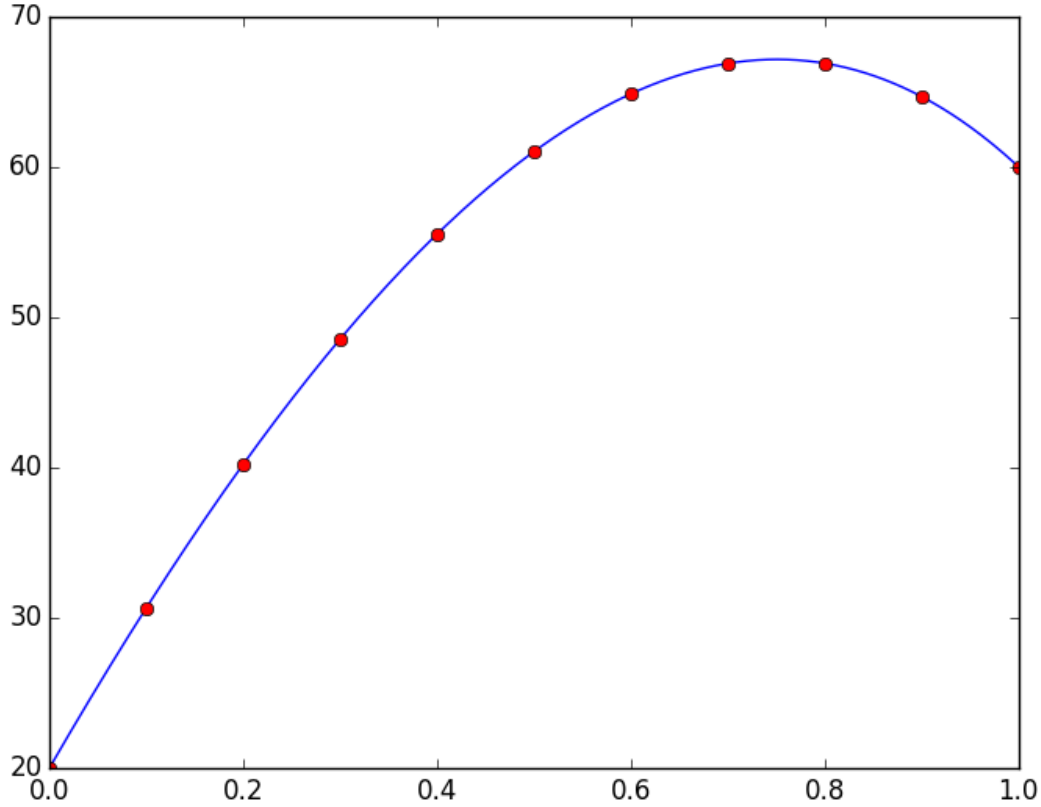
Solution:

$$u(x) = a + bx + ce^x \quad \Longrightarrow \quad u''(x) = ce^x = -100e^x \quad \Longrightarrow \quad c = -100$$
$$u(0) = a - 100 = 20 \quad \Longrightarrow \quad a = 120$$
$$u(1) = 120 + b - 100e = 60 \quad \Longrightarrow \quad b = 100e - 60$$
$$u(x) = 120 + (100e - 60)x - 100e^x$$

Steady state heat/diffusion



Steady state heat/diffusion



For more complicated equations, numerical methods must generally be used, giving approximations at discrete points.

Finite difference method

Define grid points $x_i = ih$ in interval $0 \leq x \leq 1$, where

$$h = \frac{1}{N+1}$$

So $x_0 = 0$ and $x_{N+1} = 1$ are the boundary points

and $x_1, x_2, \ldots, x_N$ are the N equally spaced interior points

Finite difference method

Define grid points $x_i = ih$ in interval $0 \leq x \leq 1$, where

$$h = \frac{1}{N+1}$$

So $x_0 = 0$ and $x_{N+1} = 1$ are the boundary points

and $x_1, x_2, \ldots, x_N$ are the N equally spaced interior points

Let $U_i \approx u(x_i)$ denote approximate solution.

We know $U_0 = \alpha$ and $U_{N+1} = \beta$ from boundary conditions.

Finite difference method

Define grid points $x_i = ih$ in interval $0 \leq x \leq 1$, where

$$h = \frac{1}{N+1}$$

So $x_0 = 0$ and $x_{N+1} = 1$ are the boundary points

and $x_1, x_2, \ldots, x_N$ are the N equally spaced interior points

Let $U_i \approx u(x_i)$ denote approximate solution.

We know $U_0 = \alpha$ and $U_{N+1} = \beta$ from boundary conditions.

Idea: replace differential equations for $u(x)$ by a system for N algebraic equations for U_i values ($i = 1, 2, \ldots, N$).

Finite difference method

$$U_i \approx u\left(x_i\right), \qquad u_{,x}\left(x_{i+\frac{1}{2}}\right) \approx \frac{U_{i+1} - U_i}{h}, \qquad u_{,x}\left(x_{i-\frac{1}{2}}\right) \approx \frac{U_i - U_{i-1}}{h}$$

Finite difference method

$$U_i \approx u\left(x_i\right), \qquad u_{,x}\left(x_{i+\frac{1}{2}}\right) \approx \frac{U_{i+1}-U_i}{h}, \qquad u_{,x}\left(x_{i-\frac{1}{2}}\right) \approx \frac{U_i-U_{i-1}}{h}$$

So we can approximate second derivative at x_i by:

$$\begin{aligned} u_{,x,x}(x_i) &\approx \frac{1}{h}\left[u_{,x}\left(x_{i+\frac{1}{2}}\right) - u_{,x}\left(x_{i-\frac{1}{2}}\right)\right] \\ &\approx \frac{1}{h}\left(\frac{U_{i+1}-U_i}{h} - \frac{U_i-U_{i-1}}{h}\right) \\ &= \frac{1}{h^2}\left(U_{i-1} - 2U_i + U_{i+1}\right) \end{aligned}$$

This gives coupled systems of N linear equations:

$$-U_{i-1} + 2U_i - U_{i+1} = h^2 f(x_i)$$

for $i = 1, 2, \ldots, N$. With $U_0 = \alpha$ and $U_{N+1} = \beta$.

Tridiagonal linear system

$$
\begin{aligned}
-\alpha + 2U_1 - U_2 &= h^2 f(x_1) && (i = 1) \\
-U_1 + 2U_2 - U_3 &= h^2 f(x_2) && (i = 2) \\
&\vdots \\
-U_{N-2} + 2U_{N-1} - U_N &= h^2 f(x_{N-1}) && (i = N-1) \\
-U_{N-1} + 2U_N - \beta &= h^2 f(x_N) && (i = N)
\end{aligned}
$$

Tridiagonal linear system

$$
\begin{aligned}
-\alpha + 2U_1 - U_2 &= h^2 f(x_1) && (i = 1) \\
-U_1 + 2U_2 - U_3 &= h^2 f(x_2) && (i = 2) \\
&\vdots \\
-U_{N-2} + 2U_{N-1} - U_N &= h^2 f(x_{N-1}) && (i = N-1) \\
-U_{N-1} + 2U_N - \beta &= h^2 f(x_N) && (i = N)
\end{aligned}
$$

For $N = 5$:

$$
\begin{bmatrix} 2 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix}
\begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \\ U_5 \end{bmatrix}
=
\begin{bmatrix} h^2 f(x_1) + \alpha \\ h^2 f(x_2) \\ h^2 f(x_3) \\ h^2 f(x_4) \\ h^2 f(x_5) + \beta \end{bmatrix}
$$

Tridiagonal linear system

$$
\begin{aligned}
-\alpha + 2U_1 - U_2 &= h^2 f(x_1) && (i = 1) \\
-U_1 + 2U_2 - U_3 &= h^2 f(x_2) && (i = 2) \\
&\vdots \\
-U_{N-2} + 2U_{N-1} - U_N &= h^2 f(x_{N-1}) && (i = N-1) \\
-U_{N-1} + 2U_N - \beta &= h^2 f(x_N) && (i = N)
\end{aligned}
$$

For $N = 5$**:**

$$
\begin{bmatrix} 2 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 2 \end{bmatrix}
\begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \\ U_5 \end{bmatrix}
=
\begin{bmatrix} h^2 f(x_1) + \alpha \\ h^2 f(x_2) \\ h^2 f(x_3) \\ h^2 f(x_4) \\ h^2 f(x_5) + \beta \end{bmatrix}
$$

General $N \times N$ system requires $O\left(N^3\right)$ flops to solve.

Tridiagonal $N \times N$ system requires $O\left(N\right)$ flops.

2D heat/diffusion equation

One-dimensional equation generalizes to

$$u_{,t}(t,x,y) = D\Big(u_{,x,x}(t,x,y) + u_{,y,y}(t,x,y)\Big) + g(t,x,y)$$

on some domain in the xy-plane with intial and boundary conditions.

We will only consider a rectangle: $0 \le x \le 1,\ \ 0 \le y \le 1.$

2D heat/diffusion equation

One-dimensional equation generalizes to

$$u_{,t}(t,x,y) = D\Big(u_{,x,x}(t,x,y) + u_{,y,y}(t,x,y)\Big) + g(t,x,y)$$

on some domain in the xy-plane with intial and boundary conditions.

We will only consider a rectangle: $0 \le x \le 1, \;\; 0 \le y \le 1$.

We consider the steady-state equation ($f(x,y) = g(x,y)/D$):

$$\begin{aligned}
-u_{,x,x}(x,y) - u_{,y,y}(x,y) &= f(x,y) && \text{in} \quad 0 < x < 1, \quad 0 < y < 1 \\
u(0,y) &= B_{\text{left}}(y) && \text{for} \quad 0 \le y \le 1 \\
u(1,y) &= B_{\text{right}}(y) && \text{for} \quad 0 \le y \le 1 \\
u(x,0) &= B_{\text{bottom}}(x) && \text{for} \quad 0 \le x \le 1 \\
u(x,1) &= B_{\text{top}}(x) && \text{for} \quad 0 \le x \le 1
\end{aligned}$$

This is a PDE in two spatial variables. (Poisson Equation)

NOTE: if $f \equiv 0$, then it is called Laplace Equation

Finite difference method

$$U_{ij} \approx u\left(x_i, y_j\right), \qquad u_{,x,x} \approx \frac{U_{i+1\,j} - 2U_{ij} + U_{i-1\,j}}{h^2}, \qquad u_{,y,y} \approx \frac{U_{i\,j+1} - 2U_{ij} + U_{i\,j-1}}{h^2}$$

Finite difference method

$$U_{ij} \approx u\left(x_i, y_j\right), \qquad u_{,x,x} \approx \frac{U_{i+1\,j} - 2U_{ij} + U_{i-1\,j}}{h^2}, \qquad u_{,y,y} \approx \frac{U_{i\,j+1} - 2U_{ij} + U_{i\,j-1}}{h^2}$$

Discrete equations:

$$4U_{ij} - U_{i-1\,j} - U_{i+1\,j} - U_{i\,j+1} - U_{i\,j-1} = h^2 f(x_i, y_j) \qquad \textit{for} \quad \begin{cases} i = 1, 2, \ldots, N \\ j = 1, 2, \ldots, N \end{cases}$$

$$U_{0\,j} = B_{\text{left}}\left(y_j\right), \qquad U_{N+1\,j} = B_{\text{right}}\left(y_j\right) \qquad \textit{for} \quad j = 1, 2, \ldots, N$$

$$U_{i\,0} = B_{\text{bottom}}\left(x_i\right), \qquad U_{i\,N+1} = B_{\text{top}}\left(x_i\right) \qquad \textit{for} \quad i = 1, 2, \ldots, N$$

On an $N \times N$ grid, where $h = 1/(N+1)$, this gives a linear system of N^2 equations in N^2 unknowns.

Finite difference method

$$U_{ij} \approx u\left(x_i, y_j\right), \qquad u_{,x,x} \approx \frac{U_{i+1\,j} - 2U_{ij} + U_{i-1\,j}}{h^2}, \qquad u_{,y,y} \approx \frac{U_{i\,j+1} - 2U_{ij} + U_{i\,j-1}}{h^2}$$

Discrete equations:

$$4U_{ij} - U_{i-1\,j} - U_{i+1\,j} - U_{i\,j+1} - U_{i\,j-1} = h^2 f(x_i, y_j) \qquad \textit{for} \quad \begin{cases} i = 1, 2, \ldots, N \\ j = 1, 2, \ldots, N \end{cases}$$

$$U_{0\,j} = B_{\text{left}}\left(y_j\right), \qquad U_{N+1\,j} = B_{\text{right}}\left(y_j\right) \qquad \textit{for} \quad j = 1, 2, \ldots, N$$

$$U_{i\,0} = B_{\text{bottom}}\left(x_i\right), \qquad U_{i\,N+1} = B_{\text{top}}\left(x_i\right) \qquad \textit{for} \quad i = 1, 2, \ldots, N$$

On an $N \times N$ grid, where $h = 1/(N+1)$, this gives a linear system of N^2 equations in N^2 unknowns.

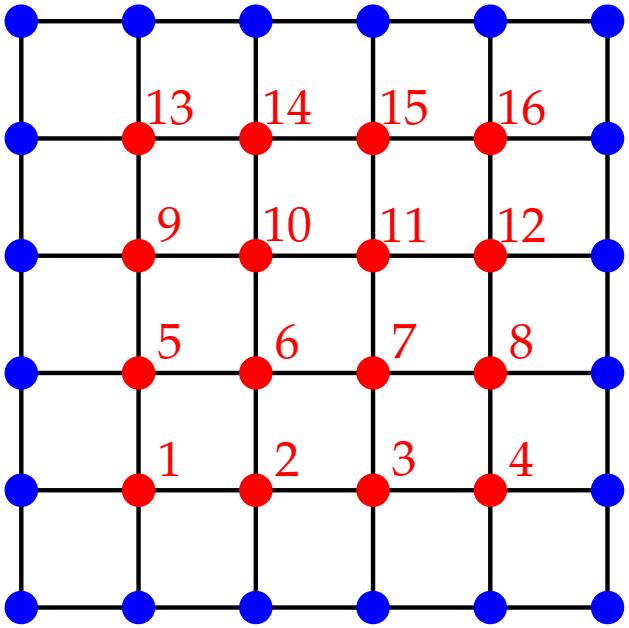
Matrix is $N^2 \times N^2$:

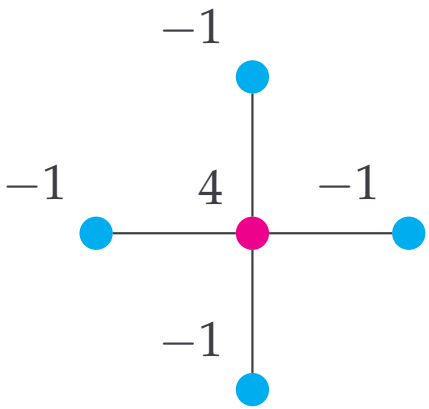
e.g., on 100 by 100 grid, matrix is $10,000 \times 10,000$.

Contains $(10,000)^2 = 100,000,000$ entries

Matrix is sparse: each row has at most 5 nonzeros out of N^2 entries!
($\approx 50,000$ non-zeros) But structure is no longer tridiagonal.

Simple Iteration Methods

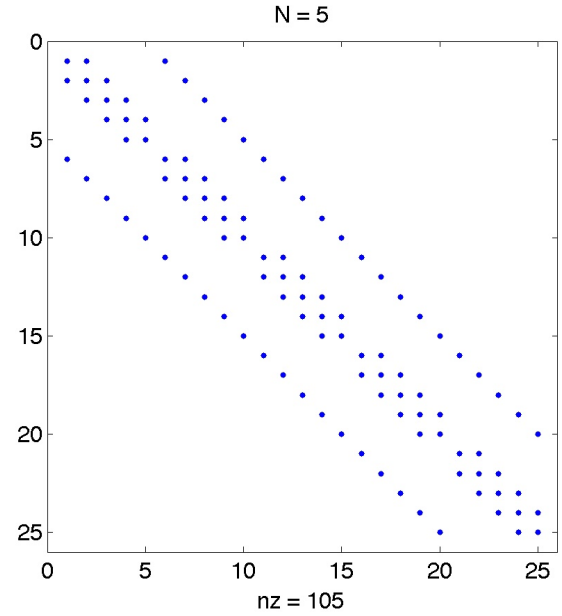


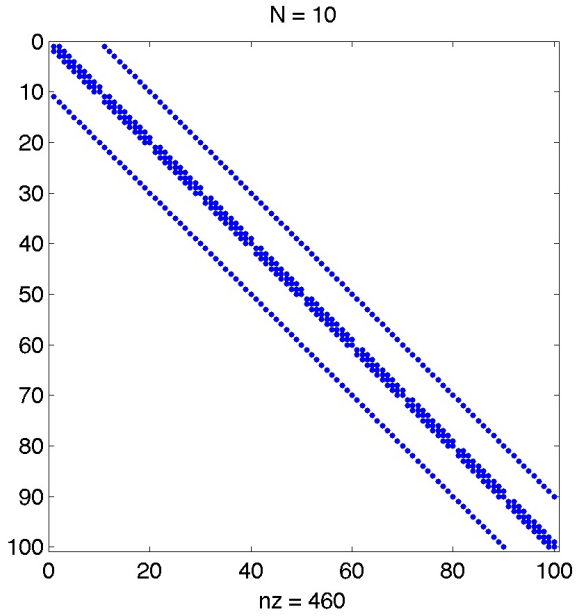


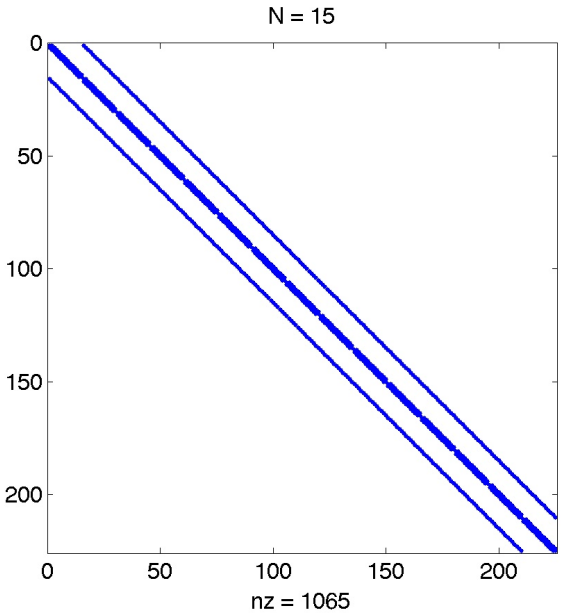
Matrix has block tridiagonal structure:

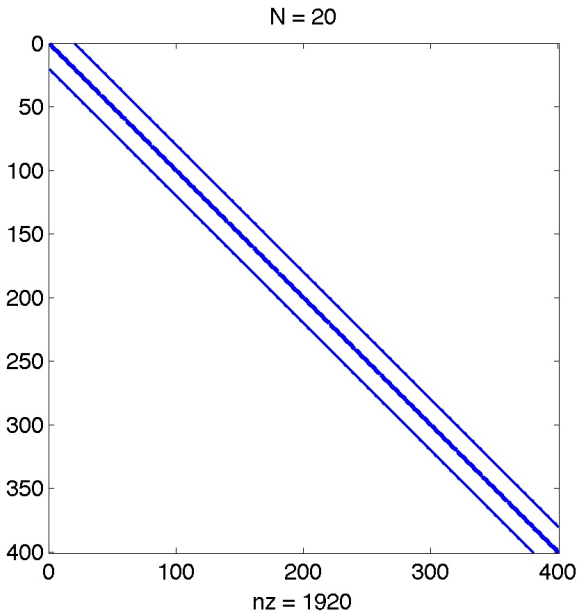
$$A = \frac{1}{h^2}\begin{bmatrix} T & -I & & \\ -I & T & -I & \\ & -I & T & -I \\ & & -I & T \end{bmatrix}, \quad T = \begin{bmatrix} 4 & -1 & & \\ -1 & 4 & -1 & \\ & -1 & 4 & -1 \\ & & -1 & 4 \end{bmatrix}, \quad I = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix}.$$

Finite difference method









Simple Iteration Methods

Direct vs. iterative methods

Direct methods for $A\mathbf{x} = \mathbf{b}$:

1. General $A \in \mathbb{R}^{m \times m}$: PLU factorization
   Dense: $\mathcal{O}(m^3)$ flops, Banded with bandwidth w: $\mathcal{O}(w^2 m)$ flops
2. SPD $A \in \mathbb{R}^{m \times m}$: Cholesky factorization ($A = LL^T$)
   Dense: $\mathcal{O}(m^3)$ flops, Banded with bandwidth w: $\mathcal{O}(w^2 m)$ flops

Direct vs. iterative methods

Direct methods for $A\mathbf{x} = \mathbf{b}$:

1. General $A \in \mathbb{R}^{m \times m}$: PLU factorization
   Dense: $\mathcal{O}(m^3)$ flops, Banded with bandwidth w: $\mathcal{O}(w^2 m)$ flops
2. SPD $A \in \mathbb{R}^{m \times m}$: Cholesky factorization ($A = LL^T$)
   Dense: $\mathcal{O}(m^3)$ flops, Banded with bandwidth w: $\mathcal{O}(w^2 m)$ flops

Difficulties:

1. Must complete all steps of GE, cannot take less steps and consider it an approximation.
2. For solving PDE in d dimensions: $m = N^d$, $\therefore PLU = \mathcal{O}(N^{3d})$

Direct vs. iterative methods

Direct methods for $A\mathbf{x} = \mathbf{b}$:

1. General $A \in \mathbb{R}^{m\times m}$: PLU factorization
   Dense: $\mathcal{O}(m^3)$ flops, Banded with bandwidth w: $\mathcal{O}(w^2 m)$ flops
2. SPD $A \in \mathbb{R}^{m\times m}$: Cholesky factorization ($A = LL^T$)
   Dense: $\mathcal{O}(m^3)$ flops, Banded with bandwidth w: $\mathcal{O}(w^2 m)$ flops

Difficulties:

1. Must complete all steps of GE, cannot take less steps and consider it an approximation.
2. For solving PDE in d dimensions: $m = N^d$, $\therefore PLU = \mathcal{O}(N^{3d})$

Alternative: iterative methods (i.e., make a guess, then iterative to improve guess). Stopping iteration before full convergence, gives an approximate solution.

Simple iteration methods (aka matrix splitting methods)

Split matrix into two parts:

$$A\mathbf{x} = \mathbf{b}, \quad \text{let} \quad A = S - T, \quad S, T \in \mathbb{R}^{m \times m}$$
$$S\mathbf{x} = T\mathbf{x} + \mathbf{b}$$

Turn this into an iteration:

$$S\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{b}$$

Idea: want S to be "close" to A, but need S to be easily invertible.

$$\mathbf{x}^{(k+1)} = S^{-1}\left(T\mathbf{x}^{(k)} + \mathbf{b}\right) = S^{-1}T\mathbf{x}^{(k)} + S^{-1}\mathbf{b}$$

Simple iteration methods (aka matrix splitting methods)

Split matrix into two parts:

$$A\mathbf{x} = \mathbf{b}, \quad \text{let} \quad A = S - T, \quad S, T \in \mathbb{R}^{m \times m}$$
$$S\mathbf{x} = T\mathbf{x} + \mathbf{b}$$

Turn this into an iteration:

$$S\mathbf{x}^{(k+1)} = T\mathbf{x}^{(k)} + \mathbf{b}$$

Idea: want S to be "close" to A, but need S to be easily invertible.

$$\mathbf{x}^{(k+1)} = S^{-1}\left(T\mathbf{x}^{(k)} + \mathbf{b}\right) = S^{-1}T\mathbf{x}^{(k)} + S^{-1}\mathbf{b}$$

Two standard versions:

1. Jacobi iteration:

$$S = \text{diagonal part of } A, \quad T = \text{-(off-diagonal part of } A)$$

2. Gauss-Seidel iteration:

$$S = \text{lower triangular part (including main)}, \quad T = \text{-(upper triangular part)}$$

Jacobi iteration for 1D Poisson

i^{th} equation in linear system:

$$-U_{i-1} + 2U_i - U_{i+1} = h^2 f(x_i)$$

Jacobi iteration:

$$-U_{i-1}^{(k)} + 2U_i^{(k+1)} - U_{i+1}^{(k)} = h^2 f(x_i) \quad \Longrightarrow \quad U_i^{(k+1)} = \frac{1}{2}\left[U_{i-1}^{(k)} + U_{i+1}^{(k)} + h^2 f(x_i)\right]$$

Jacobi iteration for 1D Poisson

i^{th} equation in linear system:

$$-U_{i-1} + 2U_i - U_{i+1} = h^2 f(x_i)$$

Jacobi iteration:

$$-U_{i-1}^{(k)} + 2U_i^{(k+1)} - U_{i+1}^{(k)} = h^2 f(x_i) \quad \Longrightarrow \quad U_i^{(k+1)} = \frac{1}{2}\left[U_{i-1}^{(k)} + U_{i+1}^{(k)} + h^2 f(x_i)\right]$$

A single iteration with boundary conditions:

$$U_1^{(k+1)} = \frac{1}{2}\left[\alpha + U_2^{(k)} + h^2 f(x_1)\right]$$

for $\quad i = 2 : (N-1)$

$$\quad U_i^{(k+1)} = \frac{1}{2}\left[U_{i-1}^{(k)} + U_{i+1}^{(k)} + h^2 f(x_i)\right]$$

end

$$U_N^{(k+1)} = \frac{1}{2}\left[U_{N-1}^{(k)} + \beta + h^2 f(x_N)\right]$$

Jacobi iteration for 1D Poisson

NOTE: the matrices A, S, and T are never actually formed!

NOTE: the flop count for 1 iteration is $\mathcal{O}(N)$ (cheap!)

Stopping criteria:

$$\delta := \frac{\|\mathbf{U}^{(k+1)} - \mathbf{U}^{(k)}\|}{\|\mathbf{U}^{(k+1)}\|}$$

$$\textbf{if} \quad \delta < \texttt{TOL} \quad \textbf{then} \quad \textbf{STOP}$$

NOTE: the total flop count for the Jacobi iteration is $\mathcal{O}(KN)$ where K is the number of iterations.

Jacobi iteration for 1D Poisson

NOTE: the matrices A, S, and T are never actually formed!

NOTE: the flop count for 1 iteration is $\mathcal{O}(N)$ (cheap!)

Stopping criteria:

$$\delta := \frac{\|\mathbf{U}^{(k+1)} - \mathbf{U}^{(k)}\|}{\|\mathbf{U}^{(k+1)}\|}$$

$$\textbf{if} \quad \delta < \texttt{TOL} \quad \textbf{then} \quad \textbf{STOP}$$

NOTE: the total flop count for the Jacobi iteration is $\mathcal{O}(KN)$ where K is the number of iterations.

How many iterations do we need?

Need to analyze the convergence properties of the Jacobi iteration method.

Matrix splitting as fixed point iteration

Simple iteration scheme:

$$\mathbf{x}^{(k)} = S^{-1}T\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

$$\text{Let} \quad G := S^{-1}T \quad \Longrightarrow \quad \mathbf{x}^{(k)} = G\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

Matrix splitting as fixed point iteration

Simple iteration scheme:

$$\mathbf{x}^{(k)} = S^{-1}T\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

$$\text{Let} \quad G := S^{-1}T \quad \Longrightarrow \quad \mathbf{x}^{(k)} = G\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

Exact solution to $A\mathbf{x} = \mathbf{b}$:

$$S\mathbf{x} = T\mathbf{x} + \vec{b} \quad \Longrightarrow \quad \mathbf{x} = S^{-1}T\mathbf{x} + S^{-1}\mathbf{b} \quad \Longrightarrow \quad \mathbf{x} = G\mathbf{x} + S^{-1}\mathbf{b}$$

Matrix splitting as fixed point iteration

Simple iteration scheme:

$$\mathbf{x}^{(k)} = S^{-1}T\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

$$\text{Let} \quad G := S^{-1}T \quad \Longrightarrow \quad \mathbf{x}^{(k)} = G\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

Exact solution to $A\mathbf{x} = \mathbf{b}$:

$$S\mathbf{x} = T\mathbf{x} + \vec{b} \quad \Longrightarrow \quad \mathbf{x} = S^{-1}T\mathbf{x} + S^{-1}\mathbf{b} \quad \Longrightarrow \quad \mathbf{x} = G\mathbf{x} + S^{-1}\mathbf{b}$$

Error in iteration:

$$\text{Let} \quad \mathbf{e}^{(k)} = \mathbf{x} - \mathbf{x}^{(k)}$$

$$\Longrightarrow \quad \mathbf{e}^{(k)} = G\mathbf{e}^{(k-1)} = G^2\mathbf{e}^{(k-2)} = G^3\mathbf{e}^{(k-3)} = \ldots = G^k\mathbf{e}^{(0)}$$

Matrix splitting as fixed point iteration

Simple iteration scheme:

$$\mathbf{x}^{(k)} = S^{-1}T\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

$$\text{Let} \quad G := S^{-1}T \quad \Longrightarrow \quad \mathbf{x}^{(k)} = G\mathbf{x}^{(k-1)} + S^{-1}\mathbf{b}$$

Exact solution to $A\mathbf{x} = \mathbf{b}$:

$$S\mathbf{x} = T\mathbf{x} + \vec{b} \quad \Longrightarrow \quad \mathbf{x} = S^{-1}T\mathbf{x} + S^{-1}\mathbf{b} \quad \Longrightarrow \quad \mathbf{x} = G\mathbf{x} + S^{-1}\mathbf{b}$$

Error in iteration:

$$\text{Let} \quad \mathbf{e}^{(k)} = \mathbf{x} - \mathbf{x}^{(k)}$$

$$\Longrightarrow \quad \mathbf{e}^{(k)} = G\mathbf{e}^{(k-1)} = G^2\mathbf{e}^{(k-2)} = G^3\mathbf{e}^{(k-3)} = \ldots = G^k\mathbf{e}^{(0)}$$

Recall: spectral decomposition

$$G = R\Gamma R^{-1} \quad \text{where} \quad \Gamma = \operatorname{diag}(\gamma_1, \gamma_2, \ldots, \gamma_N)$$

$$\text{where} \quad |\gamma_1| \geq |\gamma_2| \geq |\gamma_3| \geq \ldots |\gamma_N|$$

$$G^k = R\Gamma^k R^{-1} = R \begin{bmatrix} \gamma_1^k & & & \\ & \gamma_2^k & & \\ & & \ddots & \\ & & & \gamma_N^k \end{bmatrix} R^{-1} \quad \Longrightarrow \quad \text{convergence if} \quad |\gamma_1| < 1$$

Amplification matrix for Jacobi on 1D Poisson

$$A = \begin{bmatrix} 2 & -1 & & & \\ -1 & 2 & -1 & & \\ & \ddots & \ddots & \ddots & \\ & & -1 & 2 & -1 \\ & & & -1 & 2 \end{bmatrix}, \quad S = 2I, \quad T = S - A$$

$$G = S^{-1}T = I - S^{-1}A = I - \frac{1}{2}A$$

Therefore, G and A have the same eigenvectors and the eigenvalues of G are related in a simple way to the eigenvalues of A:

$$\gamma_i = I - \frac{1}{2}\lambda_i$$

What are the eigenvalues of A? (skip justification)

$$\lambda_1 = 2\left(1 - \cos(\pi h)\right) \quad \Longrightarrow \quad \gamma_1 = \cos(\pi h) \approx 1 - \frac{\pi^2}{2}h^2 + \mathcal{O}(h^4)$$

Therefore, $|\gamma_1| < 1$ for all $h > 0$. However $|\gamma_1| \to 1$ as $h \to 0$.

Therefore, slower convergence as $h \to 0$.

Amplification matrix for Jacobi on 1D Poisson

How many iterations are needed for $\|\mathbf{e}^{(k)}\| \approx \varepsilon \|\mathbf{e}^{(0)}\|$?

$$\gamma_1^K = \varepsilon \quad \Longrightarrow \quad K = \frac{\log(\varepsilon)}{\log(\gamma_1)}$$

How to choose ε? Solution to 1D Poisson equation is $\mathcal{O}(h^2)$ accurate, therefore take $\varepsilon = ch^2$.

$$K = \frac{\log(ch^2)}{\log(\gamma_1)} = \frac{\log(c) + 2\log(h)}{\log(\gamma_1)}$$

$$\log(\gamma_1) \approx \log\left(1 - \frac{\pi^2}{2}h^2\right) \approx -\frac{\pi^2}{2}h^2, \quad \textbf{Recall:}\, h = \frac{1}{N+1}$$

$$K \approx \frac{\log(c) - 2\log(N+1)}{-\frac{\pi^2}{2}(N+1)^{-2}} \approx \frac{4}{\pi^2}(N+1)^2 \log(N+1) = \mathcal{O}(N^2 \log N)$$

Amplification matrix for Jacobi on 1D Poisson

How many iterations are needed for $\|\mathbf{e}^{(k)}\| \approx \varepsilon \|\mathbf{e}^{(0)}\|$?

$$\gamma_1^K = \varepsilon \quad \Longrightarrow \quad K = \frac{\log(\varepsilon)}{\log(\gamma_1)}$$

How to choose ε? Solution to 1D Poisson equation is $\mathcal{O}(h^2)$ accurate, therefore take $\varepsilon = ch^2$.

$$K = \frac{\log(ch^2)}{\log(\gamma_1)} = \frac{\log(c) + 2\log(h)}{\log(\gamma_1)}$$

$$\log(\gamma_1) \approx \log\left(1 - \frac{\pi^2}{2}h^2\right) \approx -\frac{\pi^2}{2}h^2, \quad \textbf{Recall:}\, h = \frac{1}{N+1}$$

$$K \approx \frac{\log(c) - 2\log(N+1)}{-\frac{\pi^2}{2}(N+1)^{-2}} \approx \frac{4}{\pi^2}(N+1)^2 \log(N+1) = \mathcal{O}(N^2 \log N)$$

Therefore, total flop count is $\mathcal{O}(N^3 \log N)$!!!

This is not even remotely competitive with $\mathcal{O}(N)$ GE on a tridiagonal system.

In $d > 1$ dimensions, the method is more competitive, but far from optimal:

total flop count on Poisson problem in d dimensions: $\mathcal{O}(N^{2+d} \log N)$

Factor 2 improvement with Gauss-Seidel

i^{th} equation in linear system:

$$-U_{i-1} + 2U_i - U_{i+1} = h^2 f(x_i)$$

Gauss-Seidel iteration:

$$-U_{i-1}^{(k+1)} + 2U_i^{(k+1)} - U_{i+1}^{(k)} = h^2 f(x_i) \quad \Longrightarrow \quad U_i^{(k+1)} = \frac{1}{2}\left[U_{i-1}^{(k+1)} + U_{i+1}^{(k)} + h^2 f(x_i)\right]$$

A single iteration with boundary conditions:

$$U_1^{(k+1)} = \frac{1}{2}\left[\alpha + U_2^{(k)} + h^2 f(x_1)\right]$$

for $\quad i = 2 : (N-1)$

$$U_i^{(k+1)} = \frac{1}{2}\left[U_{i-1}^{(k+1)} + U_{i+1}^{(k)} + h^2 f(x_i)\right]$$

end

$$U_N^{(k+1)} = \frac{1}{2}\left[U_{N-1}^{(k+1)} + \beta + h^2 f(x_N)\right]$$

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi1D/main.c:

```c
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include "vector.h"

int main(int argc, char* argv[])
{
   // Functions needed
   void usage(const char* prog_name);
   void WriteSoln(const double alpha,
                  const double beta,
                  const vector* x,
                  const vector* U,
                  const int NumIters,
                  const char* filename);
   double Iterate(const double alpha,
                  const double beta,
                  const vector* rhs,
                  const vector* Uold,
                  vector* U);
   double func(const double x);
```

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi1D/main.c:

```
    // Input the number of grid points
    if (argc != 2) { usage(argv[0]); }
    const int N = strtol(argv[1], NULL, 10);
    if (N<0) { usage(argv[0]); }
    const double h = 1.0/((double)(N+1));
    const double h2 = pow(h,2);

    // Dimension vectors
    vector U    = new_vector(N);
    vector Uold = new_vector(N);
    vector x    = new_vector(N);
    vector rhs  = new_vector(N);
    int NumIters = 0;
    const int MaxIters = 10000;
    const double TOL = 1.0e-8;

    // Boundary conditions
    const double alpha = 20.0;
    const double beta = 60.0;
```

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi1D/main.c:

```
    // Initial guess
    for (int i=1; i<=N; i++)
    {
       const double xtmp = ((double)i)*h;
       vget(x,i)    = xtmp;
       vget(Uold,i) = alpha + xtmp*( beta - alpha );
       vget(rhs,i)  = h2*func(xtmp);
    }
    WriteSoln(alpha,beta,&x,&Uold,NumIters,"Jacobi0.data");

    // Iterate
    int mstop = 0;
    while( mstop==0 )
    {
       NumIters = NumIters+1;
       const double error = Iterate(alpha,beta,&rhs,&Uold,&U);
       if (error < TOL || NumIters==MaxIters)
       { mstop = 1; }
       else
       { copy_vector(&U,&Uold); }
       printf(" NumIters = %i, error = %23.15e\n",
              NumIters,error);
    }
```

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi1D/main.c:

```
    WriteSoln(alpha,beta,&x,&U,
              NumIters,"Jacobi1.data");

    delete_vector(&U);
    delete_vector(&Uold);
    delete_vector(&x);
    delete_vector(&rhs);

    return 0;
}

double func(const double x)
{
    return 100.0*exp(x);
}

void usage(const char *prog_name)
{
    fprintf(stderr,"usage: %s <N>\n",prog_name);
    fprintf(stderr,"   N should be positive\n");
    exit(1);
}
```

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi1D/main.c:

```
void WriteSoln(const double alpha,
               const double beta,
               const vector* x,
               const vector* U,
               const int NumIters,
               const char* filename)
{
   const int N = U->size;
   FILE* file = fopen(filename,"w");

   fprintf(file,"%i\n",NumIters);
   fprintf(file,"%24.15e %24.15e\n",0.0,alpha);
   for (int i=1; i<=N; i++)
   {
      const double xtmp = vgetp(x,i);
      const double Utmp = vgetp(U,i);
      fprintf(file,"%24.15e %24.15e\n",xtmp,Utmp);
   }
   fprintf(file,"%24.15e %24.15e\n",1.0,beta);

   fclose(file);
}
```

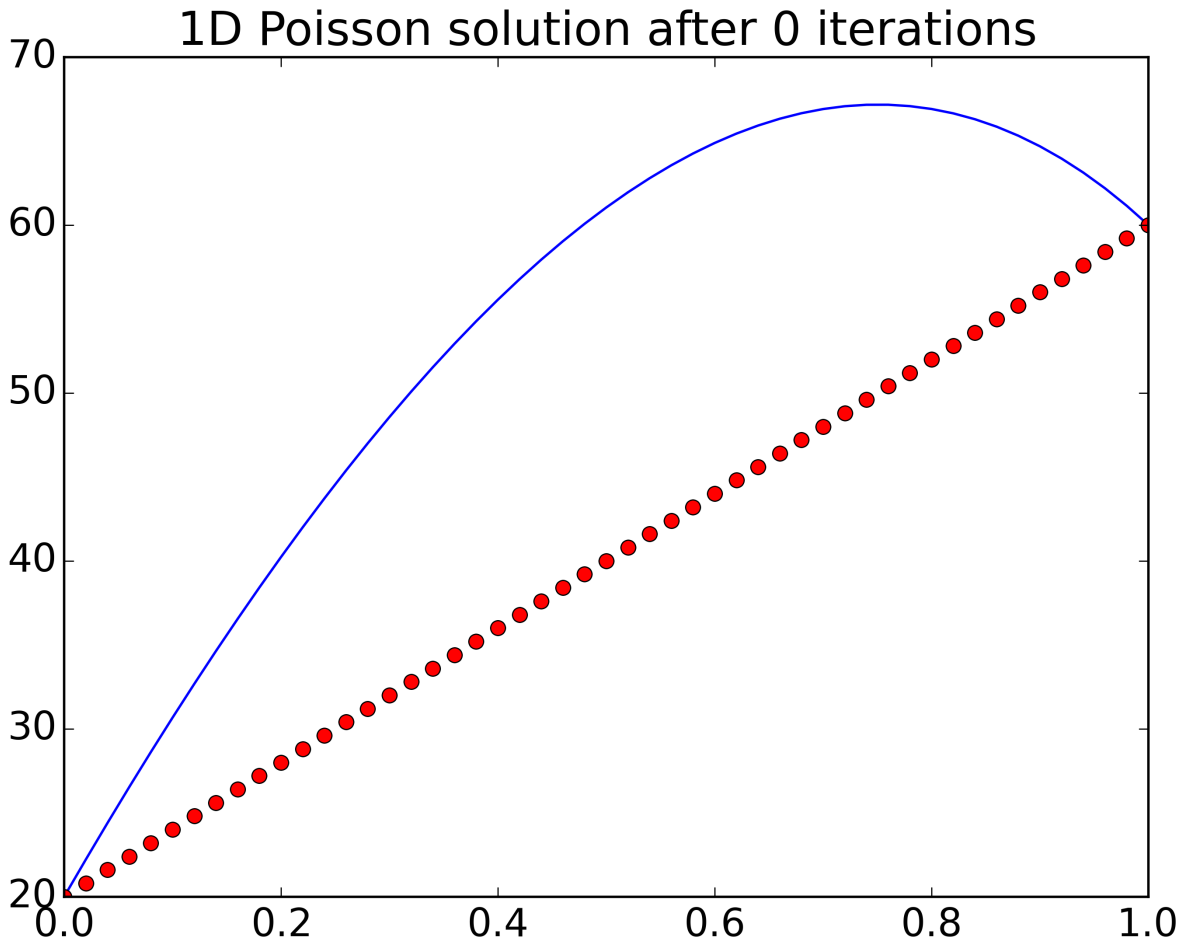
Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi1D/main.c:

```
double Iterate(const double alpha,const double beta,
               const vector* rhs,const vector* Uold,
               vector* U)
{
   const int N = U->size; const int Nm1 = N-1;
   double error = 0.0; double normU=pow(alpha,2)+pow(beta,2);

   vgetp(U,1) = 0.5*( vgetp(Uold,2) + alpha + vgetp(rhs,1) );
   error += pow(vgetp(U,1)-vgetp(Uold,1),2);
   normU += pow(vgetp(U,1),2);
   for (int i=2; i<=Nm1; i++)
   {
      vgetp(U,i) = 0.5*( vgetp(Uold,i+1) + vgetp(Uold,i-1)
                         + vgetp(rhs,i) );
      error += pow(vgetp(U,i)-vgetp(Uold,i),2);
      normU += pow(vgetp(U,i),2);
   }
   vgetp(U,N) = 0.5*( beta + vgetp(Uold,N-1) + vgetp(rhs,N) );
   error += pow(vgetp(U,N)-vgetp(Uold,N),2);
   normU += pow(vgetp(U,N),2);

   return sqrt(error/normU);
}
```

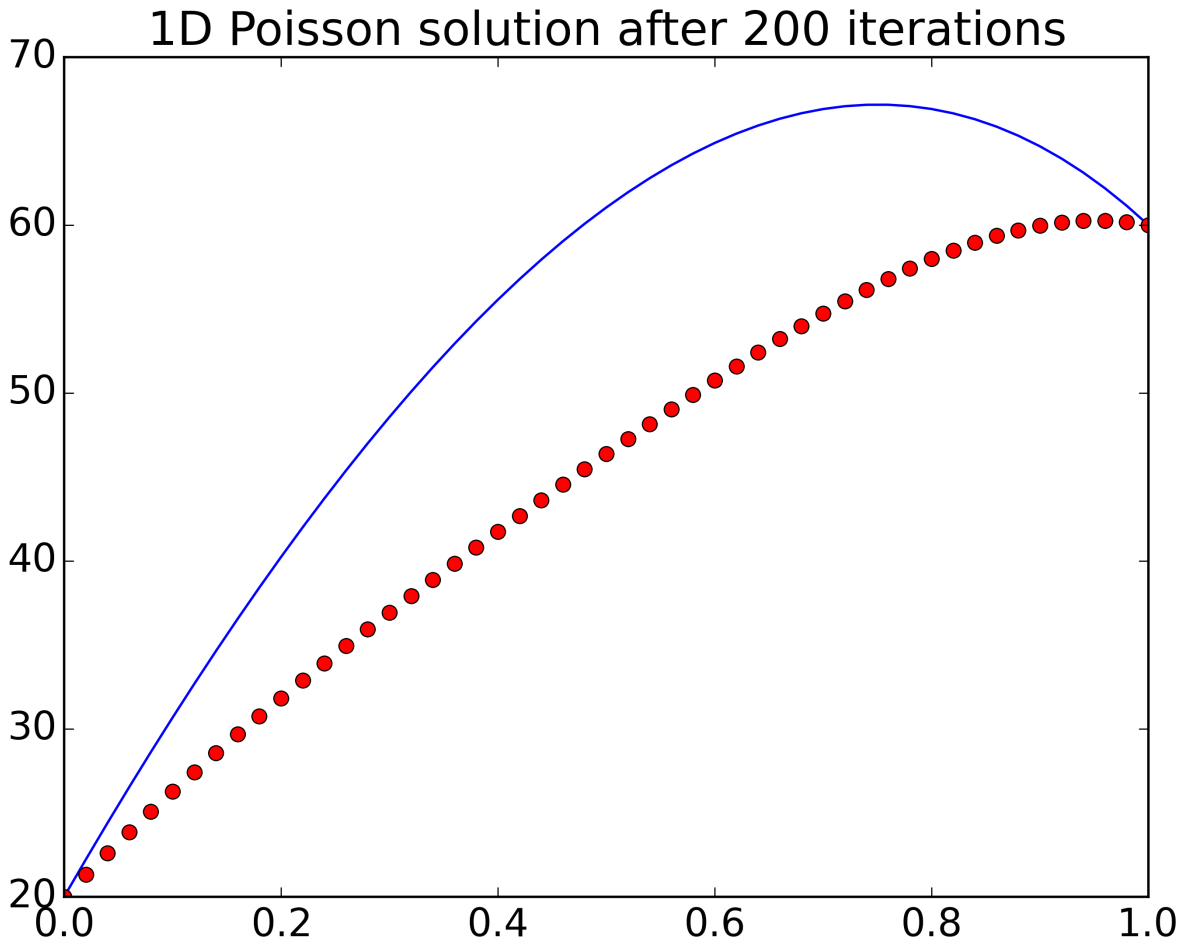
Jacobi iteration code

```
$ main.exe 49
$ python plot_soln.py
```



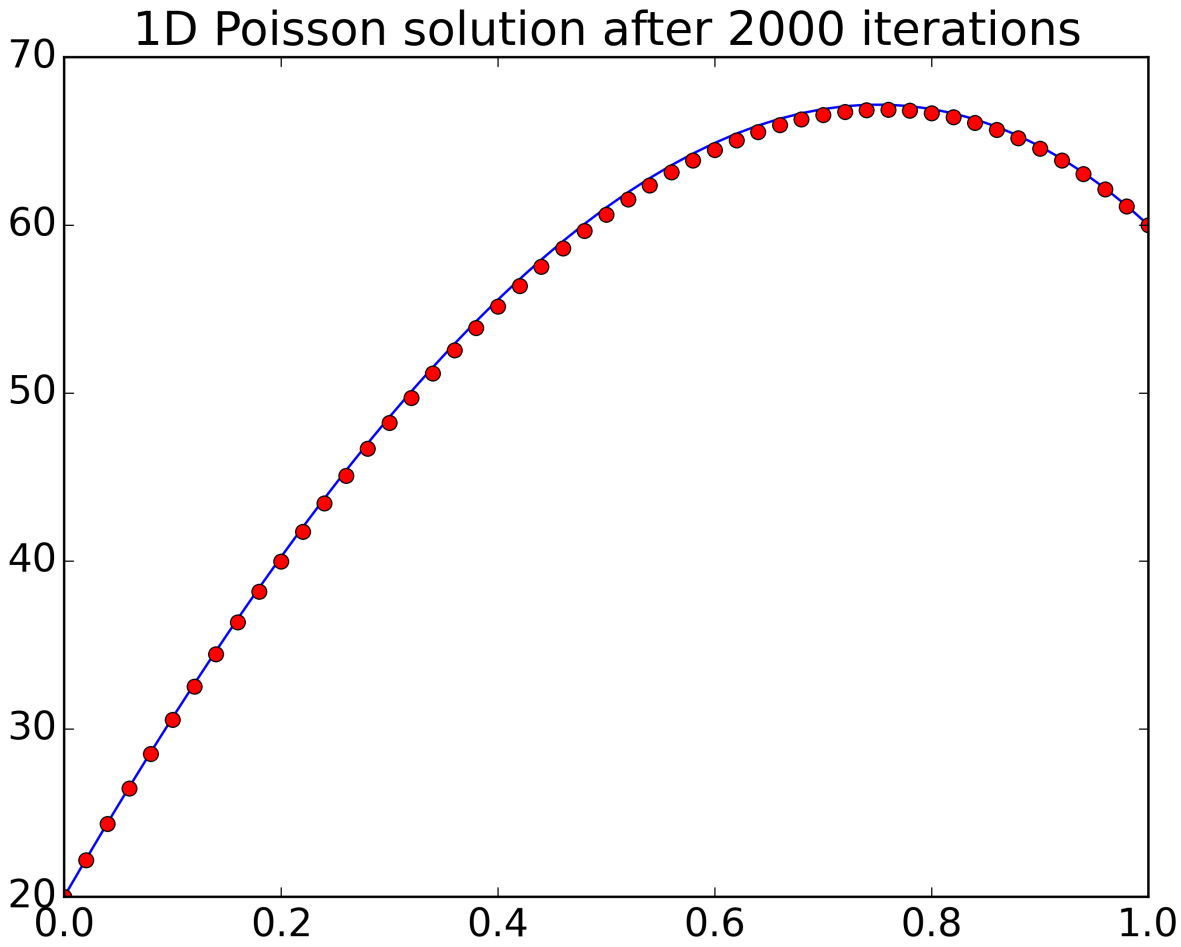
Jacobi iteration code

```
$ main.exe 49
$ python plot_soln.py
```



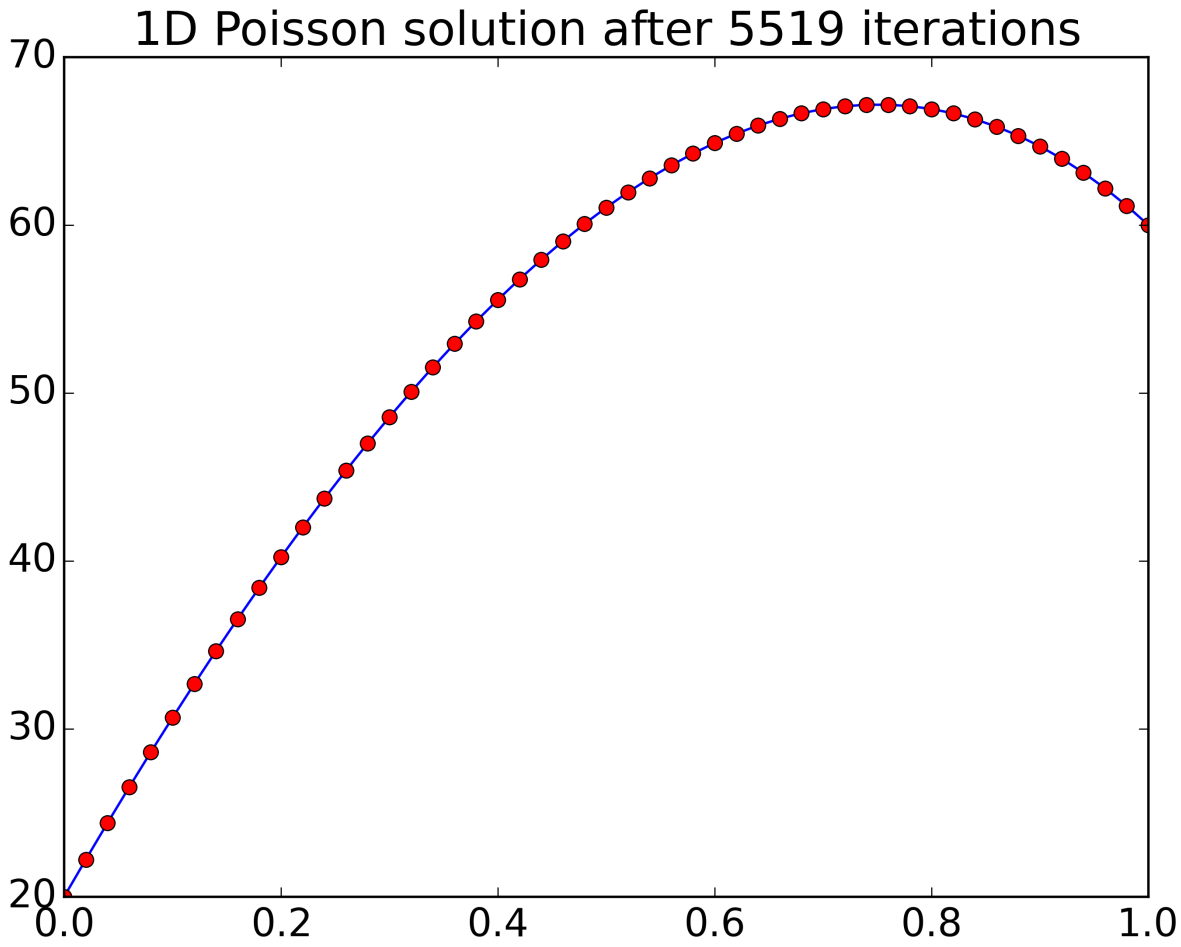
Jacobi iteration code

```
$ main.exe 49
$ python plot_soln.py
```



Jacobi iteration code

```
$ main.exe 49
$ python plot_soln.py
```



Jacobi Iteration for 2D Poisson

$$U_{11}^{(k+1)} = \frac{1}{4}\left[B_{\text{left}}(y_1) + U_{2\,1}^{(k)} + U_{1\,2}^{(k)} + B_{\text{bottom}}(x_1) + h^2 f(x_1, y_1)\right]$$

$$U_{N1}^{(k+1)} = \frac{1}{4}\left[U_{N-1\,1}^{(k)} + B_{\text{right}}(y_1) + U_{N\,2}^{(k)} + B_{\text{bottom}}(x_N) + h^2 f(x_N, y_1)\right]$$

$$U_{1N}^{(k+1)} = \frac{1}{4}\left[B_{\text{left}}(y_N) + U_{2\,N}^{(k)} + B_{\text{top}}(x_1) + U_{1\,N-1}^{(k)} + h^2 f(x_1, y_N)\right]$$

$$U_{NN}^{(k+1)} = \frac{1}{4}\left[U_{N-1\,N}^{(k)} + B_{\text{right}}(y_N) + B_{\text{top}}(x_N) + U_{N\,N-1}^{(k)} + h^2 f(x_N, y_N)\right]$$

for $j = 2 : (N-1)$

$$U_{1j}^{(k+1)} = \frac{1}{4}\left[B_{\text{left}}(y_j) + U_{2\,j}^{(k)} + U_{1\,j+1}^{(k)} + U_{1\,j-1}^{(k)} + h^2 f(x_1, y_j)\right]$$

$$U_{N\,j}^{(k+1)} = \frac{1}{4}\left[U_{N-1\,j}^{(k)} + B_{\text{right}}(y_j) + U_{N\,j+1}^{(k)} + U_{N\,j-1}^{(k)} + h^2 f(x_N, y_j)\right]$$

end

Jacobi Iteration for 2D Poisson

for $i = 2 : (N-1)$

$$U_{i1}^{(k+1)} = \frac{1}{4}\left[U_{i-1\,1}^{(k)} + U_{i+1\,1}^{(k)} + U_{i\,2}^{(k)} + B_{\text{bottom}}(x_i) + h^2 f(x_i, y_1)\right]$$

$$U_{iN}^{(k+1)} = \frac{1}{4}\left[U_{i-1\,N}^{(k)} + U_{i+1\,N}^{(k)} + B_{\text{top}}(x_i) + U_{i\,N-1}^{(k)} + h^2 f(x_i, y_N)\right]$$

end

for $i = 2 : (N-1)$

  for $j = 2 : (N-1)$

$$U_{ij}^{(k+1)} = \frac{1}{4}\left[U_{i-1\,j}^{(k)} + U_{i+1\,j}^{(k)} + U_{i\,j+1}^{(k)} + U_{i\,j-1}^{(k)} + h^2 f(x_i, y_j)\right]$$

  end

end

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi2D/main.c:

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include "matrix.h"

int main(int argc, char* argv[])
{
   // Functions needed
   void usage(const char* prog_name);
   void WriteSoln(const matrix* x,
                  const matrix* y,
                  const matrix* U,
                  const int NumIters,
                  const char* filename);
   double Iterate(const matrix* x,
                  const matrix* y,
                  const matrix* rhs,
                  const matrix* Uold,
                  matrix* U);
   double func(const double x,
               const double y);
```

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi2D/main.c:

```
    // Dimension vectors
    matrix U    = new_matrix(N,N);
    matrix Uold = new_matrix(N,N);
    matrix x    = new_matrix(N,N);
    matrix y    = new_matrix(N,N);
    matrix rhs  = new_matrix(N,N);
    int NumIters = 0;
    const int MaxIters = 100000;
    const double TOL = 1.0e-6;

    // Initial guess
    for (int i=1; i<=N; i++)
       for (int j=1; j<=N; j++)
       {
          const double xtmp = ((double)i)*h;
          const double ytmp = ((double)j)*h;
          mget(x,i,j)    = xtmp;
          mget(y,i,j)    = ytmp;
          mget(Uold,i,j) = 0.0;
          mget(rhs,i,j)  = h2*func(xtmp,ytmp);
       }
    WriteSoln(&x,&y,&Uold,NumIters,"Jacobi2D0.data");
```

Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi2D/main.c:

```
    // Iterate
    int mstop = 0;
    while( mstop==0 )
    {
       NumIters = NumIters+1;
       const double error = Iterate(&x,&y,&rhs,&Uold,&U);
       if (error < TOL || NumIters==MaxIters)
       { mstop = 1; }
       else
       { copy_matrix(&U,&Uold); }
       printf(" NumIters = %i, error = %23.15e\n",
              NumIters,error);
    }
    WriteSoln(&x,&y,&U,NumIters,"Jacobi2D1.data");

    delete_matrix(&U);
    delete_matrix(&Uold);
    delete_matrix(&x);
    delete_matrix(&y);
    delete_matrix(&rhs);

    return 0;
}
```

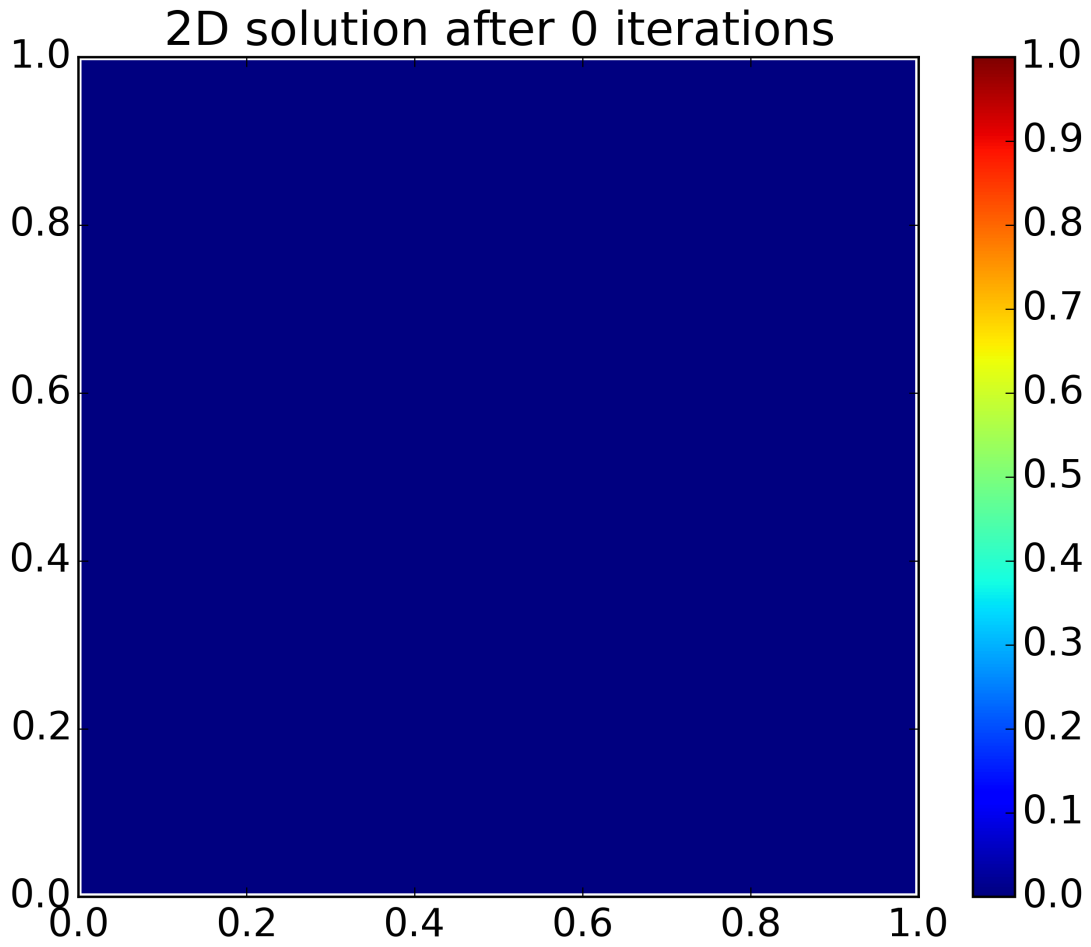
Jacobi iteration code

$ISUHPC/lectures/lecture22/codes/Jacobi2D/main.c:

```
double func(const double x, const double y)
{ return 0.0; }

double Bleft(const double y)
{ return 0.0; }

double Bright(const double y)
{   if (y<=0.3 || y>=0.7)
    {   return 0.0; }
    else
    {   return 1.0; }
}

double Bbottom(const double x)
{ return 0.0; }

double Btop(const double x)
{   if (x<=0.3 || x>=0.7)
    {   return 0.0; }
    else
    {   return 1.0; }
}
```

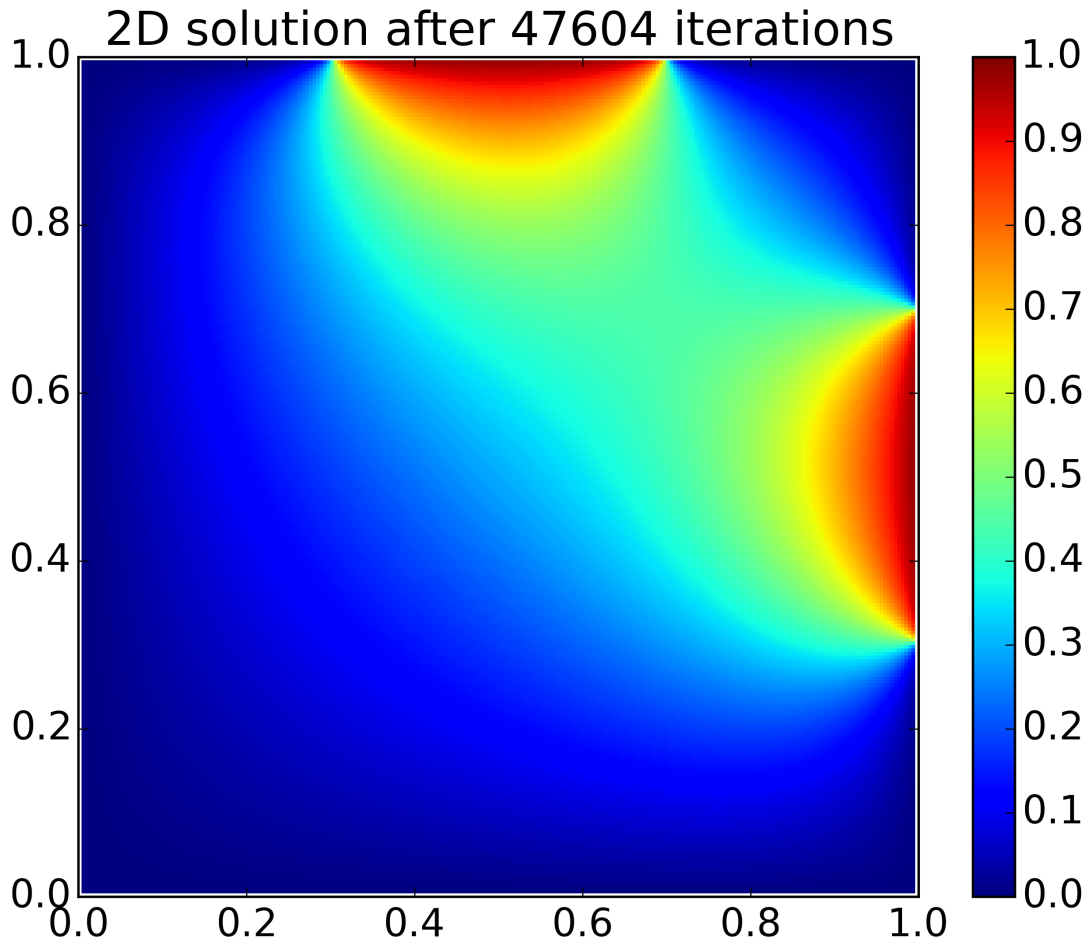
Jacobi iteration code

```
$ main.exe 239
$ python plot_soln.py
```



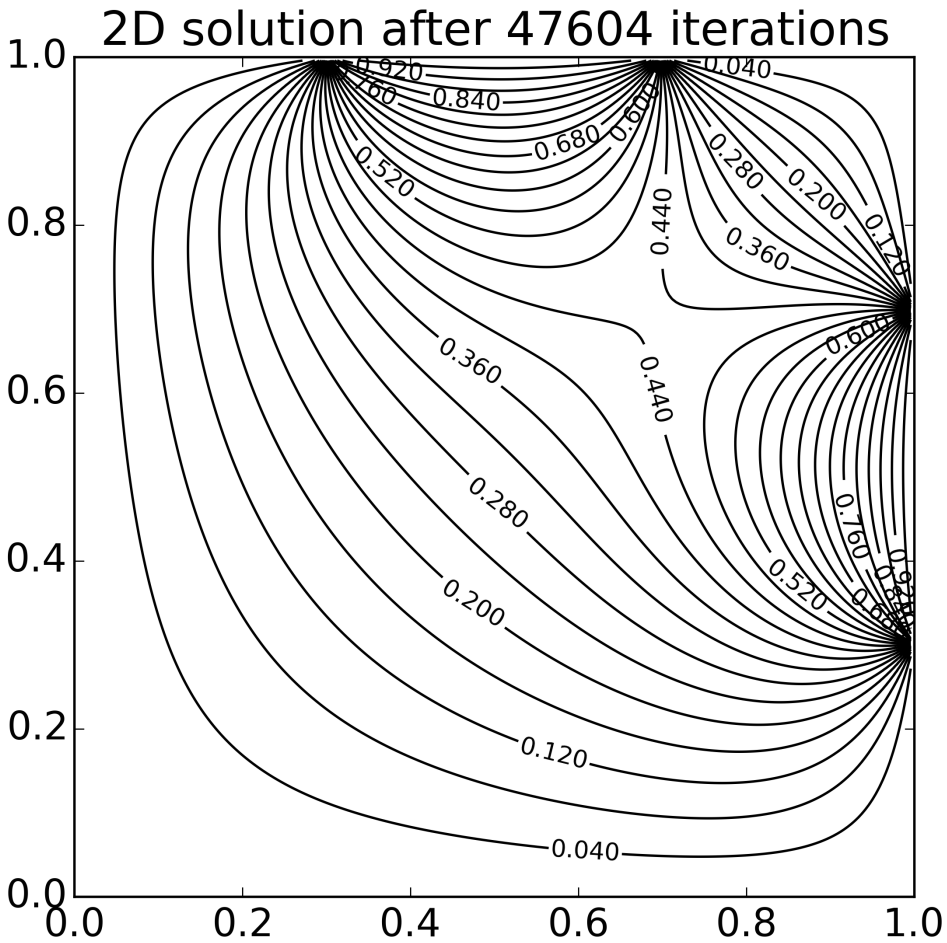
Jacobi iteration code

```
$ main.exe 239
$ python plot_soln.py
```



Jacobi iteration code

```
$ main.exe 239
$ python plot_soln.py
```



Lab Assignment

- Monte Carlo method for computing π.
- MPI implementation vs OpenMP implementation: using coarse-grain parallelism.
- Submit source codes, screenshots and update Git.